No. ..... Date. .....

① Data Structures & Algorithms

Algorithm

- A recipe
- A finite, well defined set of steps for solving a problem.
- Can be more than one algorithm to solve a problem.

A good algorithm should ⟶
- be correct
- be composed of a finite number of well defined steps.
- Produce output
- be able to terminate in finite time.
- be efficient.

Program

- An implementation of an algorithm using a programming language.

Good program should be ⟶
- able to execute correctly.
- able to execute efficiently (within the resource constraints)
- easy to read and understand.
- easy to maintain.

Pseudocode

- A kind of structured English for specifying algorithms.
- Means Fake code.
- Cannot be compiled nor executed.
- No strict syntax rules.
- Easy to read and understand.
- Programming language independent.
- Designed for humans, not machines.

EDUCATION IS THE BEST INVESTMENT

CamScanner

No. .....

Date. .....

Efficiency of an algorithm.

- Amount of memory and time needed to run a program written with a particular algorithm.
- Used to pick the most appropriate one when there are multiple algorithms exist for the same problem.
- Determining efficiency
    - Performance measurement
    - Performance analysis.

No. ..... Date. .....

Write compilable, executable, smallest program you can write in C++

```
int main () {
    return 0;
}
```

C:\> echo %ERRORLEVEL%

0

C:\> echo hello

hello

C:\> echo ERRORLEVEL

ERRORLEVEL

test.cpp

test.exe

C:\> test.exe

↓

OS calls main

↓

main() calls other ~~fun~~ functions

↓

main() returns exist status to OS.

Antivirus system ⟶ Scan.exe test.exe

0 - no virus

1 - virus found ⟶ Anti system ⟶ clean.exe test.exe

0 - cleaned

1 - clean failed.

Antivirus sys ⟶ quarantine.exe test.exe

Logical operators

Short circuit	Full evaluation (Both are evaluated)
&&	&
||	|
!	~

if (expression 1 && expression 2)

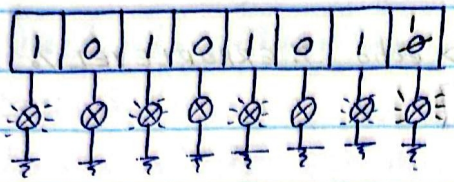
if (expression 1 || expression 2)

EDUCATION IS THE BEST INVESTMENT

CamScanner

No. ..... Date. .....

Hhy Full evaluation



Change last bit without affecting others.
Bit to Bit manipulation.

10101010 | 00000001 ⟶ 10101011

* All global variables are automatically initialize.

Uninitialized values ⟶ Garbage collector.
State in between local & global.

No. Date.

② C++

* Return code 136 → Division by zero error.

* Static - Automatically initialize First time.

```
#include <iostream>
using namespace std;
void counter()
{
    int count; // uninitialized        static int count; //initialized once.
    count++;
    count << count << "\n";
}
void polluteMemory()
{
    int garbage = 999;
}
int main()
{
    polluteMemory();
    counter();
    counter();
    return 0;
}
```

```
1000
1001
1002
```

* Use initialized variables. Otherwise we get various values because of garbage values.

EDUCATION IS THE BEST INVESTMENT

CamScanner

No. ..... Date. .....

small change.

```
{ if (num[i] < 0)
    continue ;
  cout << . . . . .
}
```

1	1
9	3
16	4

```
#include <iostream>
#include <cmath>
using namespace std;
int main()
{
    int num[] = { 1, -4, 9, 16, -25};
    int size = 5;
    for (int i=0;  i< size;  i++)
        cout << num[i] << "\t" << sqrt(num[i]) << "\n";
    return 0;
}
```

1	1
-4	-nan (not a number)
9	3
10	4
-25	-nan

(1)

```
int main()
{
    int pins[] = { 1234, 3456, 5678, 7890, 9012};
    int size = 5, target = 5678;
    for (int i=0;  i< size;  i++)
    {
        if (pins[i] == target)
        {
            cout << "PIN is valid " << "\n";
            break;
        }
        else
            cout << " Checking\n" ;
    }
    return 0;
}
```

In if condition
2 lines → { }
1 line → ( )

Checking
Checking

EDUCATION IS THE BEST INVESTMENT

No. ..... Date. .....

* break ⟶ Exit from the loop.
But not from all the loops Only from inner loop.

(ii)

```
int size = 5, target = 5678, attempts = 3;
For (int i=0; i < attempts; i++)
{
  for(int j=0; j < size; j++)
  {
    if (pins [j] == target)
    {
      ;
      ;
      !
```

Checking
Checking
PIN is valid
} repeat 3

break ⟶ Exit from inner loop ~~durrank~~.
return 0 ⟶ Exit from entire program.
return ⟶ Exit from inner loop & outer loop.

(iii)

```
{
cout << "PIN is valid" << "\n";
  return 0;
}
```

Checking
Checking
PIN is valid

EDUCATION IS THE BEST INVESTMENT

CamScanner

No. .....

```
void printSqrt (int n)
{
   if (n <0)
      return ; // stop function and exit
   cout << n << "\t" << sqrt (n) << "\n";
}
int main ()
{
   int num [ ] = {1, -4, 9, 16, -25};
   int size = 5;
   For (int i=0;   i<size;   i++)
     int printSqrt (num[i]);
   return 0;
}
```

1	1
9	3
16	4

No. ..... Date. .....

```
(5) void checkPins (int p[] int s, int a, int t)
{
(6) for (int i=0 ; i<a ; i++)
    {
(7)  for (int j=0 ; j<s ; j++)
     {
(8)   if (p[j] == t)
      {
(11)      cout << " PIN valid " ;
(12)      return ;
(9)   } else
(10)      cout << " Checking" ;
      }
    }
}
(1) int main ( ) {
(2)  int pin[ ] = { 123 , 132 , 141 , 512 , 213 } ;
(3)  int attempts = 3 , size = 5 , target = 141 ;
(4)  checkPins (pin , size , attempt , target) ;
(13)    cout << " PIN found " ;
(14)  return 0 ;
}
```

No. Date.

Speciality → no memory allocation.

#define PI 3.14 ← define is useful because if we found a new value for pi we have to change multiple code lines. With define its only have to change one line.

const float pi 3.14 → same as this but this take a memory allocation → memory uses.

```
#define PI 3.14
int main()
{   float r, c;
    cout << "Enter radius";
    cin >> r;
    c = 2 * PI * r;
    cout << "Circumference is : " << c;
    return 0;
}
```

During the compilation translate PI into 3.14
No memory allocation.

No. ..... Date. .....

For variables anytime can assign values.
So accidently add values middle of the code.
That's why use constant for fixed values.

Conditional ternary operator.

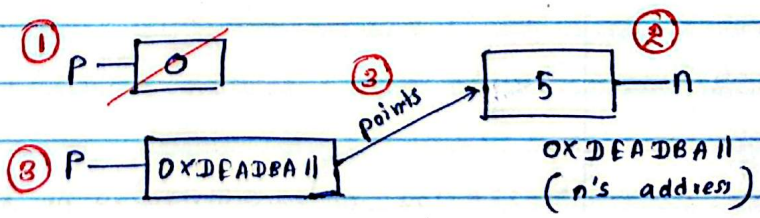
```
#include <iostream>
using namespace std;
int main ()
{
    int inCity = 1 ;  // in city 1 , out city 0
    const int speed = inCity ? 30 : 70 ;
    cout << "Speed is " << speed << " kmph\n" ;
    return 0;
}
```

Speed is 30 kmph

Pointers - It can hold memory address.

```
#include <iostream>
using namespace std;
int main () {
① int *p = 0 ;
② int n = 5 ;
③ p = &n ;
   cout << n << "\n" ;
   cout << &n << "\n" ;
   cout << p << "\n" ;
   cout << *p << "\n" ;
   return 0 ;
}
```

If the pointer contain zero it means null pointer. Pointer doesn't point anywhere.




```
5
0xDEADBA11
0xDEADBA11
5
```

EDUCATION IS THE BEST INVESTMENT

CamScanner

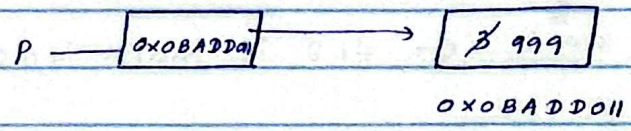
No. ..... Date. .....

Wild pointers

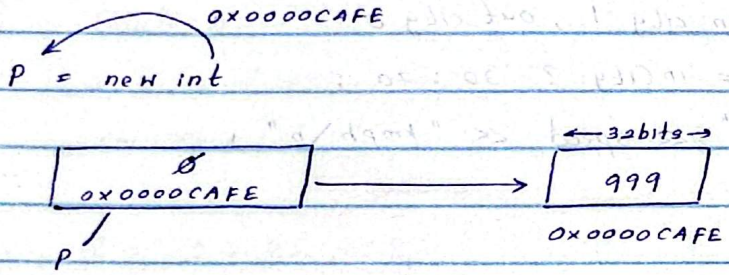
```
int * P ;
cout << * P ;      3
* P = 999 ;
```

* When we use pointers we should initialize.

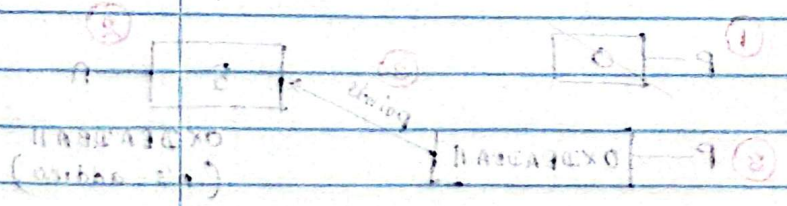
* Wild pointers can point any memory location.




```
int * P = 0
```




```
* P = 999 ;
cout << * P ;      999
cout << P ;        0x0000CAFE
```



No. Date.

③

```
int *P = new int ;
```

Four segments of memory

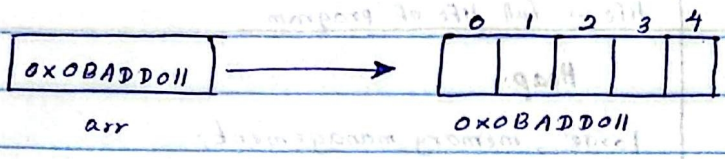
Text	Data
keep code	Keep inputs, outputs and global variable. life → full life of program.
Stack	**Heap**
Small, few mega bytes. fast, they are created When func call destroy after func. Memory allocations are automatic	Large, memory management. Allocation, deallocation slow life time decide by programmer.

```
int *P = new int (999);
cout << *P ;  999
*P = 1000 ;
cout << *P ;  1000
```

If We declared a static variable it goes to Data.

No. Data.

```
void leakMemory() {
    int *P = new int(999);
    cout << *P;
    delete P;
    return;
}
```




```
int size;
cout << "How many elements?";
cin >> size

int *arr = new int[size];
```

delete ⟶ delete [] arr;

```
char *a = 0;
int *b = 0;
float *c = 0;
cout << sizeof(a) << "\n";
cout << sizeof(b) << "\n";
cout << sizeof(c) << "\n";
return;
```

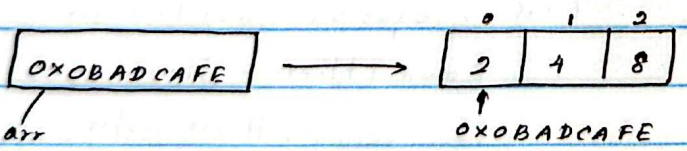
8
8
8

EDUCATION IS THE BEST INVESTMENT

CamScanner

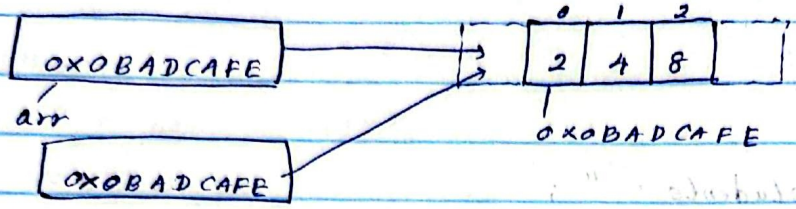
```
int n;
double sum = 0;
cout << " No of students : " ;
cin >> n;
int * marks = new int [n];
for (int i=0; i<n; i++)
{
cout << "Marks of student " << i+1 << " : ";
cin >> marks [i];
```

```
{
int n = 5;
int *P = 0;
P = &n;
cout << P << "\n";          0x7ffd4294f5f4
cout << P+1 << "\n";        0x7ffd4294f5f8
cout << P-1 << "\n";        0x7ffd4294f5f0
return 0;
}
```



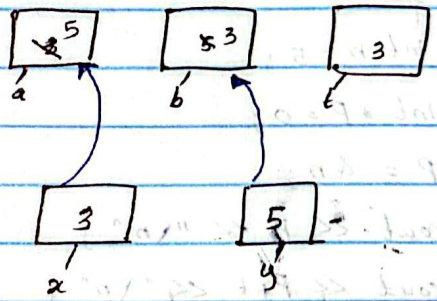

```
cout << &arr[0];
        0x0BADCAFE

cout << arr ;          0x0BADCAFE
cout << arr[0];        2
cout << *arr;          2
```


CamScanner




```
cout << *(p+1) ;        2
cout << *(p+2) ;        4
cout << *(p+3) ;        4
cout << *(p-1) ;        4
                        8
                        #.
```

```
void swap (int a, int b)
{   int t = a;
    a = b;
    b = t;
}

int main()
{
   int x = 3, y = 5;
   swap (x, y);
   cout << x << " " << y << "\n";
   return 0;
}
```



3 5

```
void swap (int *a, int *b)
{
  int t = *a;
  *a = *b;
  *b = t;
}

int main (){
  int x = 3, y = 5;
  swap (x, y);
  cout << x << " " << y << "\n";
  return 0;
}
```

5 3

Default parameters.

```
int power(int num, int expo = 2)
{
  int result = 1;
  for (int i=0; i<expo; i++)
    result = result * num;
  return result;
}

int main(){
  cout << power (2,3) << '\n';
  cout << power (2) << "\n";
  return 0;
}
```

8
4.

No. ..... Date. .....

Space Complexity

Space complexity - How much memory (RAM) a program needs while it is running.

Why space complexity

- To know whether enough memory is available.
- To compare two programs
- To know the biggest problem a computer can solve.

Components of space complexity

- Instruction space
- Data space
- Environmental stack space.

(01) Instruction

The memory needed to store the program itself. Usually fixed.

(02) Data space

The memory needed for variables and constants.

int - 4 bytes
double - 8 bytes
char - 1 byte

arrays

int arr[100];

1 int → 4 bytes

100 integers → 100 × 4 = 400 bytes.

(03) Environmental stack

The memory used when functions are called.

When hello(); is called, computer stores:

- return address
- parameters
- local variables

EDUCATION IS THE BEST INVESTMENT

CamScanner

No. ..... Date. .....

Fixed space vs Variable space

Fixed : Always the same

includes → Program instructions
constants
simple variables.

int a;

No matter what input you give, they always use the same memory.

Variable : Depends on input size.

int arr[n];

if n = 10 memory = 40 bytes
n = 1000 memory = 4000 bytes.

* Memory change with input.

Total Space = Constant space + Variable space

ex: int sum (int a, int b, int c)

* only 3 integers, memory never changes.

space complexity = 0

$S(p) = C + Sp(input)$

C = Fixed memory

Sp = variable.

ex: for (i=0; i<n; i++)

n changes,

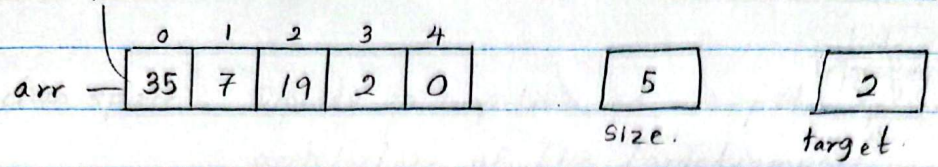
the program doesn't create more variables. Only i is used.

∴ sp(n) = 0.

variable space = 0

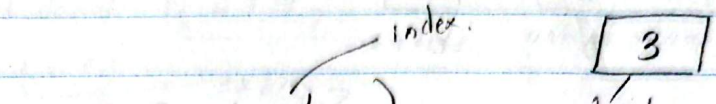
overall space complexity = O(1)

No: Date: 17/07/2026

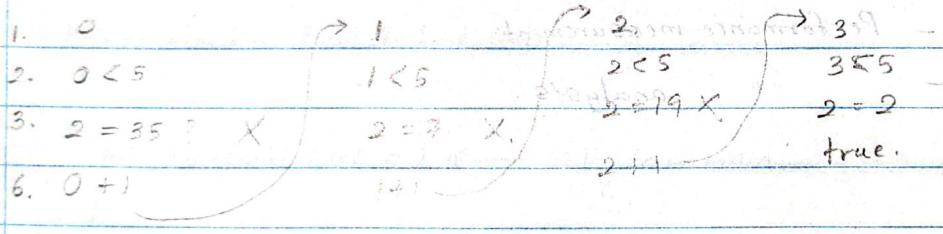


location INTEGER
outcome BOOLEAN

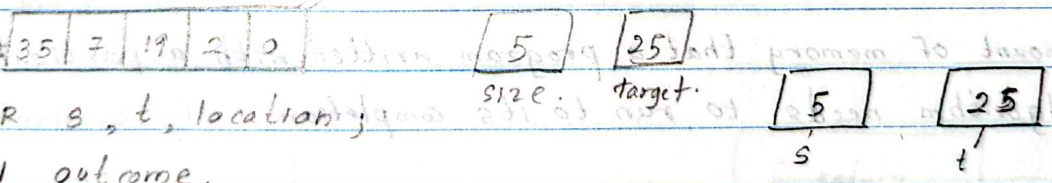
outcome → search (arr, 5, 2, location)



IF (outcome)
  PRINT location
END IF



(02)



INTEGER s, t, location;
BOOLEAN outcome.
s ← 5
t ← 25
outcome ← search (arr, s, t, location)
PRINT location.

0
0 < 5
25 = 35 X.
0 + 1

1
1 < 5
25 = 7 X
1 + 1

2
2 < 5
25 = 19 X,
2 + 1

3
3 < 5
25 = 2 X
3 + 1

4
4 < 5
25 = 0.
4 + 1

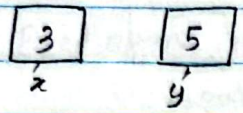
5
5 < 5 X
False

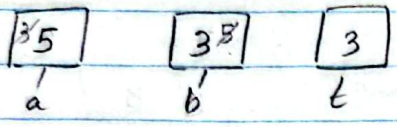
Atlas

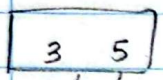
CamScanner

No: Date: __/__/____

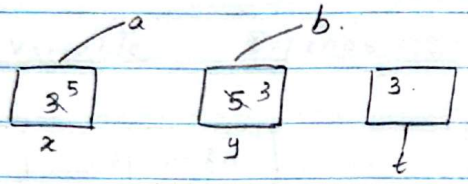
(03)







(04)



reference parameters – No local copy

slide 7

Post determination – Performance measurement

Pre " – " analysis.

Space Complexity

Amount of memory that a program written with a particular algorithm needs to run to its completion.

Why – To know whether the available memory is enough to run the program written with our algorithm.

- Consumes less memory
- Could estimate the largest problem that our algorithm can solve against the memory available.

Components

Instruction space

Data space.

Environmental stack space.

Atlas

CamScanner

No: _____ Date: __/__/____

options.
code size optimization.
execution time optimization.

Instruction space - Depends on compiler used, compiler options used, architecture of the target computer.

Data space - const double PI = 3.14; — 64 bits, named variable
int r = 7; — 32 bits
int C; — 32 bits.
} Premitive variables.

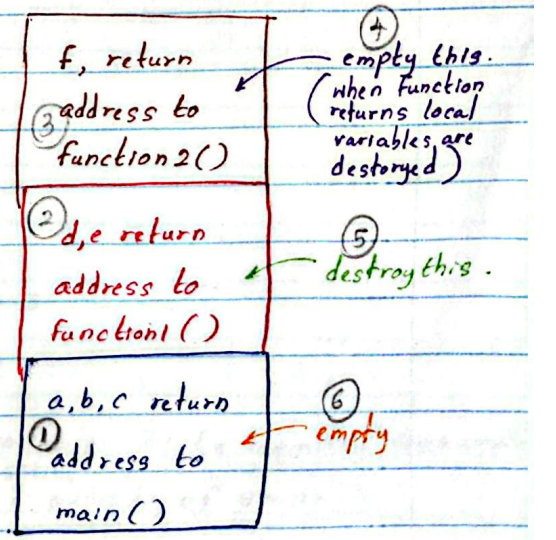
C = 2 * PI * r;
cout << C;

Environment stack space. - Memory needed for the environment stack.
A data structure that keeps information needed to resume execution of partially completed functions.

Rule of stack - First In Last Out

```
int function1 (int a)   <- call from main   (1)
{
  int b, c;
  ...
  b = function2 (c);        (6)
  ...
  return b;
}
int function2 (int d)       (2)
{
  int e;
  ...
  function3 ();             (5)
  ...
  return e;
}
void function3 ()           (3)
{ int f;
  ...
}
```

Return to main



Atlas

CamScanner

No: _____ Date: __/__/____

Space complexity of algorithm A.

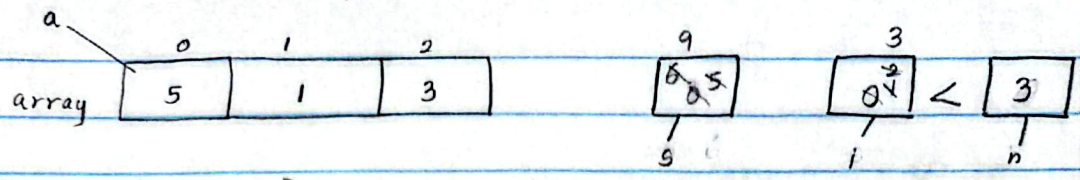
$$S(n) = C + S_A(n)$$

Where C is insensitive (constant) space contribution and S_A is the sensitive space contribution which is a function of instance characteristic n.

No. ..... Date. 24/07/2026

⑤

(01)



sum (array, 3)

Return s = 9

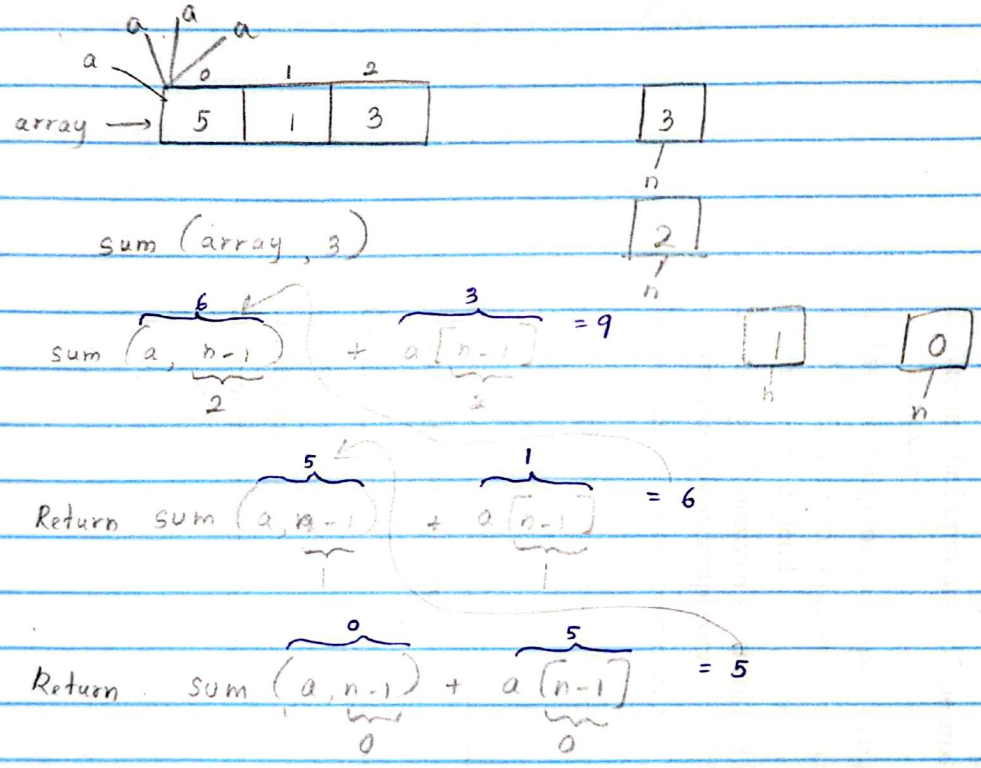
- Memory consumption of n, s, i are insensitive (premitive variables)
- array is structured variable.
- As 'a' array passed by reference its memory consumption is also insensitive to this algorithm.

$S(n) = c + 0$

$S(n) \approx 0$

* Algorithm is non recursive. So environmental stack space consumption is also insensitive.

(02)



n - premitive variable. So its memory consumption is insensitive.

a[] - declared outside of scope.

So its memory consumption is also insensitive.

environment stack space consumption is sensitive to the problem being solved.

for each function call, environment stack keeps n, a, return address of calling point.

Assuming target computer vs 32-bit machine for n - 4 bytes, a - 4 bytes, return address - 4 bytes. each function call 4+4+4 = 12 bytes He keep in environment stack

Recursion depth n+1

S(n) = 12(n+1)

9

No. ..... Date. .....

⑥

Time Complexity

Amount time a program takes to run based on the size of inputs.
It doesn't measure actual seconds.

Why ?

- Real time needs - some systems must responds instantly.
- Choose the fastest - When the multiple algorithms solve the same problem, pick the one that runs quicker.
- Avoid timeout - Ensure your program finishes within limits, especially for large data.

Factors of the time complexity

1- Input size (n) - The main
2- Hardware & Software.
3- Operation types - Arithmatic comparison

Time complexity

$$T(n) = C + T_A(n)$$

C- constant time. (not depend on input size)
$T_A(n)$ - Sensitive time (does depend on the input size)

Two approaches

01- Operation count
02- Step count.

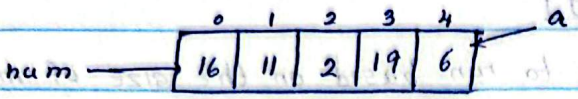
Operation count

Key operation - most time consuming operator.

* Key operation then other operation don't care.
* ~~Conti~~ Counting how many times certain operations are performed.
* Count only most important operations like comparisons, additions, multiplications.

No. ..... Date. .....

Ex: 01)



max (num, size) → return → index of where the largest integer is 3.

$1 < 5$	$2 < 5$	$3 < 5$
$16 < 11$	$16 < 2$	$16 < 19$
$c \leftarrow c+1$	$c \leftarrow c+1$	$i \leftarrow c$
$2 \leftarrow 1+1$	$3 \leftarrow 2+1$	$i \leftarrow 3$

largest one from the array.

$\underbrace{a[0]}_{16} < \underbrace{a[1]}_{11}$

$\underbrace{a[0]}_{16} < \underbrace{a[2]}_{2}$

$\underbrace{a[0]}_{16} < \underbrace{a[3]}_{19}$

$\underbrace{a[3]}_{19} < \underbrace{a[4]}_{6}$

① $i = 0$
② $c = i+1$
③ Loop ($c < n$)
④ IF ($a[i] < a[c]$)
· $i \leftarrow c$ ($19 > 16$ ✓)
⑤ END IF
⑥ $c \leftarrow c+1$
⑦ END Loop

E

No. Date.

Find key operation

① Loop ($c<n$)

* go to memory & get value of c
* go to memory & get value of n
* compare two values & see $c<n$
* IF so, give true.
* Other wise give false.

IF $(a[i] < a[c]$ → * key

go to memory & get value of i

" " " c

" " " $a[i]$

" " " $a[c]$

" " " a

compare $a[i]$ & $a[c]$ & see $a[i] < a[c]$

How many time execute

$\boxed{T(n) = n-1}$

How many times execute the key operation

$T(n)$

ex: 02) Best $n = 1$ $T_1(n) = 1$

$n = 2$ $T_2(n) = 2$

$n = 3$ $T_3(n) = 3$

$$T(n) = \frac{(1+2+3+\ldots+n)}{n}$$

$$\boxed{T(n) = \frac{1}{n} \sum_{i=1}^{n} i} = \frac{1}{n} \frac{n(n+1)}{2} = \frac{1}{2}(n+1)$$

No. ..... Date. .....

Steps out approach

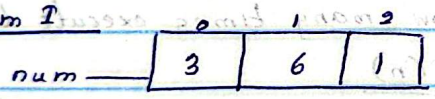
Algorithm 1

①	$s \leftarrow 0$	1
②	$c \leftarrow 0$	1
③	Loop ($c < n$)	$n+1$
④	$s \leftarrow s + a[c]$	n
⑤	$c \leftarrow c+1$	n
⑥	END Loop	
⑦	RETURN s	1, (bcz exit)
⑧	END Sum	$T(n) = 3n+4$

HOW to get step 3 time

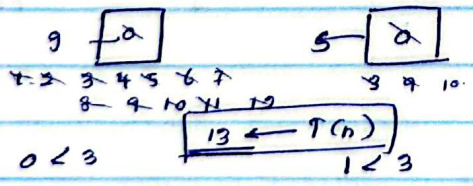
$0<0$	$n=1$	$n=2$	$n=n$
① time	$0<1$ $1<1$	$0<0$ $0<1$ $0<2$	$n+1$ times
	② times	③ times	

Program I



Size — 3 n — 3

Sum (num, size)



$0<3$	$1<3$	$2<3$	$3<3$
$s \leftarrow 0+3$	$s \leftarrow 3+6$	$s \leftarrow 9+1$	
$s \leftarrow 3$	$s \leftarrow 9$	$s \leftarrow 10$	
$c \leftarrow c+1$	$c \leftarrow 1+1$	$c \leftarrow 2+1$	
$c \leftarrow 1$	$c \leftarrow 2$	$c \leftarrow 3$	

$T(n) = 3n+4$

$T(n) = 3 \times 3 + 4$

$T(n) = 13$

EDUCATION IS THE BEST INVESTMENT

CamScanner

No. ..... Date. .....

Algorithm 2

IF (n > 0) - - -

No. ..... Date. .....

⑦ **Time Analysis**

Running Times

01- Best case running time - Time taken when the best possible thing happens.
T(n) minimum.

02- Horst case running time - Take lot of time.
T(n) maximum.

03- Average case running time - T(n) would be the average. (more complicated to analyze)

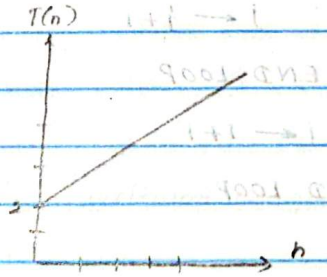
Functions of Time Complexity

* Time complexity of an algorithm is a function of n.
* $n \geq 0$
* $T(n) \geq 0$ for all n.

Linear Function

ex 01)

1.	$i \leftarrow 1$	1
2.	Loop ($i <= n$)	$n+1$
3.	$i \leftarrow i+1$	n
4.	END LOOP	



$T(n) = 2n + 2$

$y = mx + c$ - Linear line

Logarithm

02)

1.	$i \leftarrow 1$	1
2.	LOOP ($i <= n$)	$\log_2(n) + 1$
3	instruction 1	$\log_2(n)$
4	instruction 2	$\log_2(n)$
5	instruction 3	$\log_2(n)$
6	instruction 4	$\log_2(n)$
7	$i \leftarrow i+2$	$\log_2(n)$ +
8	END LOOP	

$T(n) = 6\log_2(n) + 2$

$i = 2^{\text{iterations}}$

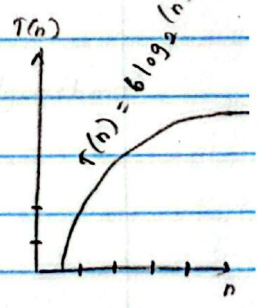
$i \leq n$

$2^{\text{iterations}} < n$

$2^{\text{iterations}} > n$

$2^{\text{maxiterations}} = n$ - Terminating point

$\text{maxiterations} = \log_2(n)$



EDUCATION IS THE BEST INVESTMENT

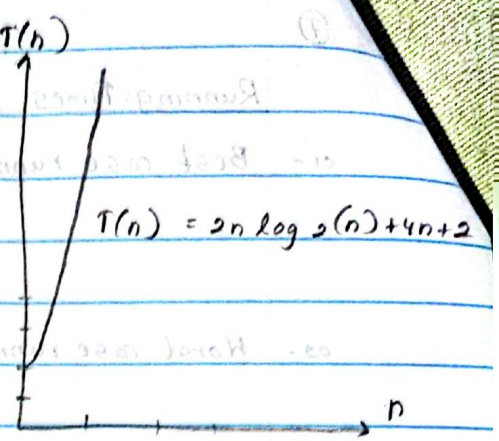
No. ..... Date. .....

Linear Logarithm

03)

1	i ← 1	1
2	LOOP (i <= n)	n+1
3	j ← 1	n
4	LOOP (j <= n)	$n(\log_2(n)+1)$
5	j ← j * 2	$n(\log_2(n))$
6	END LOOP	
7	i ← i+1	n
8	END LOOP	

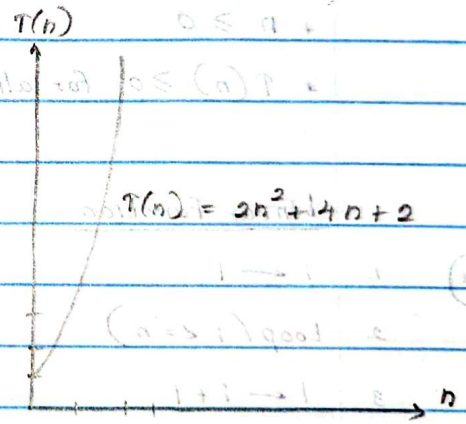
$$T(n) = 2n\log_2(n) + 4n + 2$$



Quadratic

1	i ← 1	1
2	LOOP (i <= n)	n+1
3	j ← 1	n
4	LOOP (j <= n)	n(n+1)
5	j ← j+1	n(n)
6	END LOOP	
7	i ← i+1	n
8	END LOOP	

$$T(n) = 2n^2 + 4n + 2$$



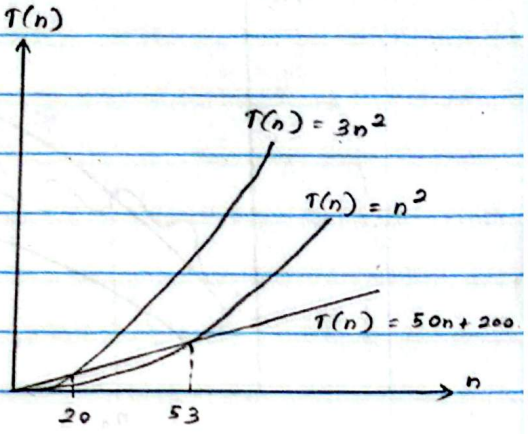
No. ..... Date. .....

Asymptotic Notation

Expresses an algorithm's running time as its growth rate by dropping the less significant terms & constant coefficients.

$T(n) = 3n^2 + 50n + 200$

$T(n) \leq 3n^2 + 50n + 200$



For all $n \geq 20 \longrightarrow 3n^2 \geq 50n + 200$

$n = 19 \longrightarrow 1083 \geq 1150$ ×.

$n = 20 \longrightarrow 1200 \geq 1200$ ✓

$n = 21 \longrightarrow 1323 \geq 1250$ ✓

For $n \geq 20$ $T(n) \leq 3n^2$ more significant.

For $n \geq 53 \longrightarrow n^2 \geq 50n + 200$

$n = 52 \longrightarrow 2704 \geq 2800$ ×

$n = 53 \longrightarrow 2809 \geq 2850$ ×

$n = 54 \longrightarrow 2916 \geq 2900$ ✓

For $n > 53$ $T(n) < n^2$ more significant. ∴ coefficient can ignore.

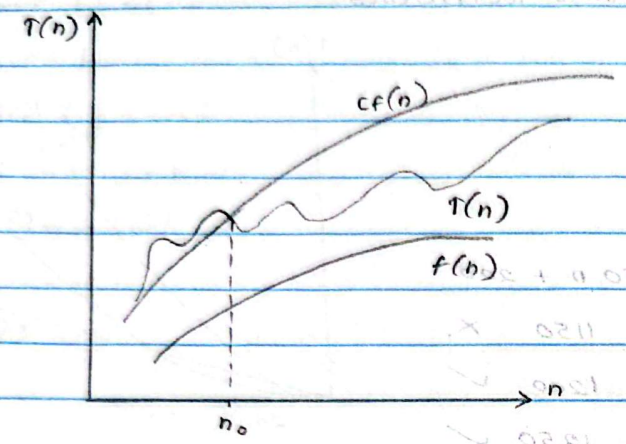
$$\lim_{n \to \infty} \left(\frac{50n + 200}{3n^2} \right) = 0$$

To find most appropriate algorithm for a problem He compare algorithms. for that He simplify like this.

No. ..... Date. .....

Big Oh Notation

Asymptotic notation for the Worst case running time of an algorithm.



For all $n \geq n_0$

$T(n) \leq cf(n)$

Where

$T(n) \geq 0$

$f(n) \geq 0$

$c \geq 0$

Definition –: For $T(n) \geq 0$ and $f(n) \geq 0$, $T(n)$ is said to be $O(f(n))$ (big Oh of F of n) if there exist positive constant c and n_0, such that $T(n) \leq cf(n)$ for all $n \geq n_0$.

(01) $T(n) = 3n + 2$

$T(n) = 3n+2 \leq 3n + n$ For all $n \geq 2$

$T(n) \leq 4n$ For all $n \geq 2$

c ← 4n → f(n)

$3n + 2 \leq 4n$ (n_0 = 2)

$n = 1 \quad 5 \leq 4 \quad \times$

$n = 2 \quad 8 \leq 8 \quad \checkmark$

$n = 3 \quad 11 \leq 12 \quad \checkmark$

$T(n) = O(f(n))$

$T(n) = O(n)$

No. ..... Date. .....

(02) $T(n) = 10n^2 + 4n + 2$

$T(n) = 10n^2 + 4n + 2 \leq 10n^2 + 4n + n$ For all $n \geq 2$

$T(n) \leq 10n^2 + 5n$ for all $n \geq 2$.

$T(n) \leq 10n^2 + n^2$ for all $n \geq 5$

$T(n) \leq 11n^2$ for all $n \geq 5$

c ↑ $f(n)$ ↑ $n_0 = 5$ ↑

$T(n) = O(f(n))$

$T(n) = O(n^2)$

$O(\log n)$ - Best

$O(n)$ - Better

$O(n^2)$ - Worst

CamScanner

No: _____ Date: __/__/____

⑧ **Data Structures**

Atomic

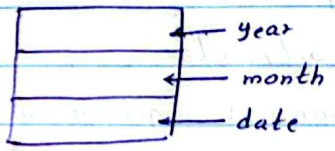
* Atomic data types - Character data types (Premitive data types)
* All data same types.
*

Composite

* Decomposable
* Consist of a set of homogeneous or heterogeneous.

ex:

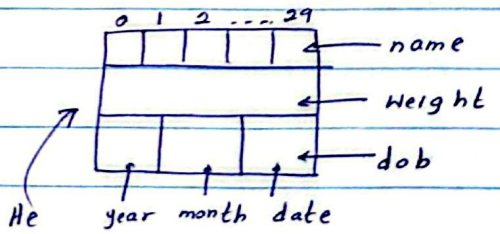
```
struct Day
{
    int year;
    int month;
    int date;
}

Day birthDay;
```



Birthday

← Composed with atomic data types.

```
Struct Person
{
    char name[30];
    int Height;
    Day dob;
};

Person He;
```

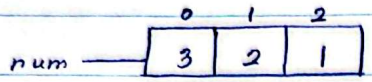


Data structure

A structure that holds a set of atomic or composite data.

No :_____ Date: __/__/____

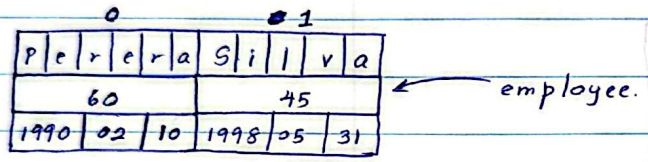
ex: (01) `int num [ ] = { 3 , 2 , 1 } ;`



get & set operations.

```
Print num [0] ;
num [2] = 5 ;
```

(02) `Person employee [ ] = { "Perera", 60 , 1990 , 02 , 10 , "Silva" , 45 , 1998 , 5 , 31 } ;`



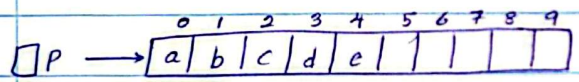
* Heterogeneous.
* 2 compartments.
* Operations : get operation `print employee [0]. name ;`
  set operation `employee [1] . Height = 57 ;`
* Both atomic & composite.
* Linear

No: _____ Date: __/__/____

Data Structures Stacks

* Stack is a linear data structure.
* additions - push
* removals - pop
* restricted to one end-(top)
* Rule - Last In First Out (LIFO)
* 2 error conditions :
  - Overflow - stack is full, but have to top. ∴ error.
  - Underflow - empty stack, but have to pop. ∴ error.
* Before push, We have to make sure stack isn't full.
  is Full ( ).
* When pop, we have to make sure stack isn't empty.
  is Empty ( )

Array based stack



We need these two variables.

top - because we always push or remove items from top.

*. top variables keep the track of top of the stack.

Create operation

Pointer to datatype

size

integer.

CamScanner

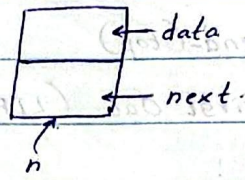
No : _____ Date: __/__/____

Test

Linked list - based stack

* Every element links to other.

```
Struct node
{
   char data;
   node  *next;
};

node n;
```



This can hold a address of similar node.

Test Algorithm (Array based)

```
ALGORITHM test()
1. create(stackSize)
2. READ(data)
3. LOOP(data ≠ endOfData AND push(data))
4. READ(data)
5. END LOOP
6. LOOP(POP data)
7. PRINT(data)
8. END LOOP
9. destroy()
10. END test.
```

size | top | S | data

5 | -1 | 5 |

P → [A | B | C | |] (indices 0 1 2 3 4)

```
ALGORITHM create(VAL s INTEGER)
1. size ← s
2. top ← -1
3. P ← ALLOCATE MEMORY DATATYPE[size]
END create
```

d = A

top	
0	A
1	B
2	C

```
ALGORITHM push(VAL d DATATYPE)
1. IF (isFull())
2. RETURN false
3. ELSE
4. top ← top + 1      -1+1 = 0
5. P[top] ← d         0+1 = 1
6. RETURN true        1+1 = 2
7. END IF
END push
```

```
ALGORITHM isFull()
1. IF (top = size-1)      -1 = 5-1 = 4
2. RETURN true            0 = 5-1 = 4, 1 = 4, 2
3. ELSE                   false. false. false.
4. RETURN false
5. END IF
END isFull
```

top	d
4	E
3	D
2	C
1	B

```
ALGORITHM pop(REF d DATATYPE)
1. IF (isEmpty())
2. RETURN false
3. ELSE
4. d ← P[top]
5. top ← top-1        4-1 = 3, 3-1 = 2, 2-1 = 1
6. RETURN true
7. END IF
END pop
```

```
ALGORITHM isEmpty()
1. IF (top = -1)          4 = -1, 3 = -1, 2 = -1
2. RETURN true            false. false. false
3. ELSE
4. RETURN false
5. END IF
END isEmpty
```

```
ALGORITHM destroy()
1. RELEASE MEMORY[] P
END destroy
```

CamScanner

Char array - last element is automatically null_character (\0)

0	1	2	3	4	5	6	7	8
K	e	l	a	n	i	y	a	\0

1. Write an algorithm to accept null terminated string of characters (text) and print it in reverse order.

```
Create stack
LOOP (data = \0)
  push (data)
LOOP (pop (data) = fail)
  pop (data)
  print (data)
```

```
ALGORITHM reverseText (REF text [] CHAR)
  S ← stack ()
  ~~ch ← text [0]~~
  i ← 0
  ch ← text [i]
  LOOP (ch ≠ '\0' AND S.push (ch))
    ~~S.push (ch)~~
    i ← i + 1
    ch ← text [i]
  END LOOP
  LOOP (S.pop (ch))
    PRINT ch
  END LOOP
END reverseText
```

This is much safer than `LOOP (ch ≠ '0')`

`S.push (ch)`

Why reverse (uses).

- undo feature.
- back button in browser

} These kind of applications implement using stack.

No. .....

Date. .....

Converting numbers

2. Write an Algorithm to accept a decimal number, convert it to binary & then print binary number.

```
ALGORITHM convert
  S ← stack()
  number ← 25
  LOOP (number ≠ 0)
  Remainder ← number % 2
  push (Remainder)
  END LOOP
  LOOP (POP(data))
   print
  END LOOP
ALGORITHM END

ALGORITHM d2b (n INTEGER)
   S ← stack()
LOOP(n>0)
bit ← n MOD 2
S.push (bit)
 n ← n/2
END LOOP
LOOP (S.pop(bit))
PINT bit
END LOOP
S.destroy()
END d2b
```

```
2 | 25
2 | 12 - 1
2 | 6  - 0
2 | 3  - 0
2 | 1  - 1
    0  - 1
```

CamScanner

Parsing data

3. Write an algorithm to accept mathematical expression & check whether there is any bracket mismatch.

$(a+b) \times c$

0	1	2	3	4	5	6	7
(	a	+	b	)	x	c	=

```
ALGORITHM bracketMismatch.
 S ← stack()
 READ (data)
Loop (data = '=')
    IF (data1 = open bracket)
    push data
    READ (data)
 IF (data = closed bracket)
    pop data 1.
 IF stackIsEmpty()
  print "No mismatch"
 ELSE
  print "Mismatch"
```

```
ALGORITHM isBracketOk (REF exp[] CHAR)
S ← stack []
i ← 0
ch ← exp[i]
LOOP (ch ≠ '=')
  IF (ch == 'c')
    s.push (ch)
  ELSE
    IF (ch == ')')
      IF (NOT s.pop(ch))
        RETURN false
      END IF
    END IF
  END IF
  i ← i+1
  ch ← exp [i]
END LOOP

IF (s.pop (ch))
  RETURN false
ELSE
  RETURN true
END IF
s.destroy()
END isBracketOk.
```

No. Date.

Data Structures Queues

Operations

~~enque()~~ enqueue() - add an item

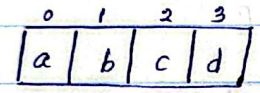
dequeue() - take out an item.

Array based queue

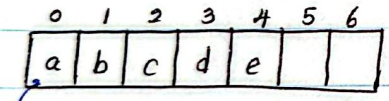
From where to dequeue is called 'front' ← Position 0. ~~id~~ index of front.

From where to enqueue is called 'rear' ← to which element enqueue (index)

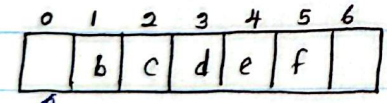
* When you add more items rear would move to right.
* When you remove more items front moves to right.



- You remove 'a' then (current front-a)
- the after remove 'a' the new front become.

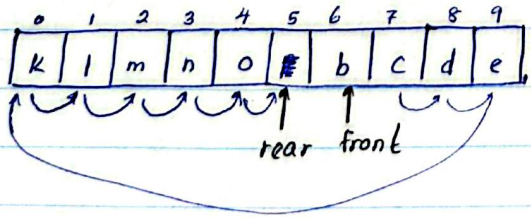
① 

count = 5 rear - 5 front - 0

② 

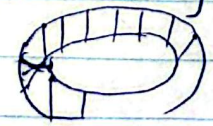
count = 5, rear - 6, front - 1

pg-7 After ~~remo~~ inserting more items.



When you imagine this as a bended and also ends are connected you will understand this.

(array is becoming full)



EDUCATION IS THE BEST INVESTMENT

CamScanner

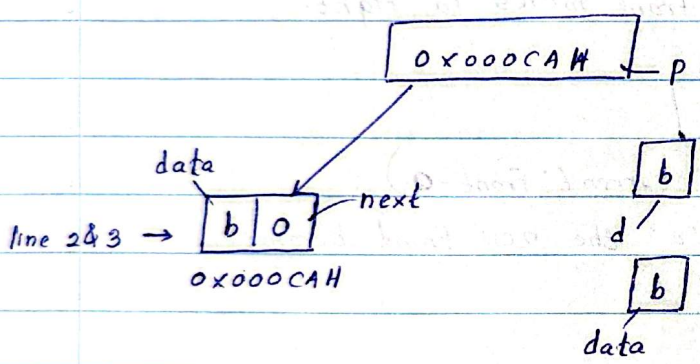
No. ..... Date. .....

* Here you have to imagine bend the array and connect two ends as a ring.

When queue is empty } rear & front - same place.
When queue is full

The circular array ⟶ (You have to imagine)

rear ← rear MOD size (circular array thanks to line no 7)



[rear → next ← d] means ; ⟶ Using rear go to next assinged.

P → data = d ;
(*P). data = d ;

No. .....

Date .....

Trees

Interaction

```
ALGORITM Countdown()
Int x
IF x > 0
   LOOP (x > 0)
      Print x
      x = x - 1
   END LOOP
ELSE
   Print "Error"
END IF
```

Recursion

```
ALGORITM countdown (VAL n INTEGER)
 IF  x > 0
      PRINT n
      countdown (n-1)
 ELSE
      RETURN
  END IF
END ALGORITM
```

CamScanner

No. ..... Date. .....

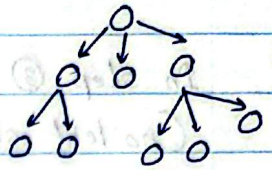
Function should accept any positive integer and return the sumation.

```
ALGORITM sum(VAL x INTEGER)
  tot = x
  IF  x > 0
     tot = tot + sum (x-1)
  ELSE
     RETURN
  END IF
  PRINT tot
END sum ()

ALGORITM sum (VAL n INTEGER)

  IF n > 0
     return  n + sum (n-1)
  ELSE
     return 0
  END IF
  END sum
```

EDUCATION IS THE BEST INVESTMENT

CamScanner

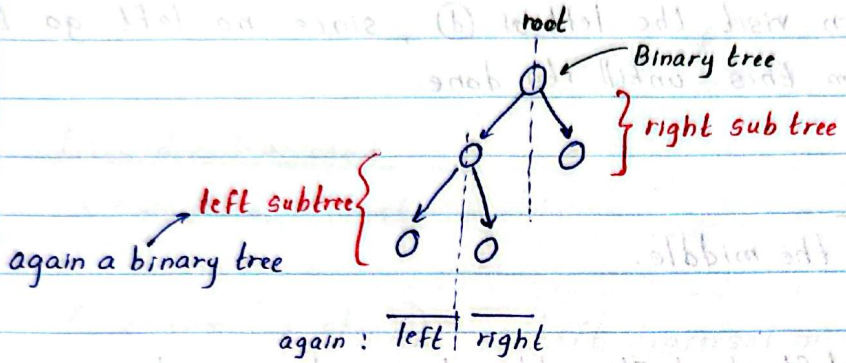
Trees



order → the max children no of the tree.

Binary Tree

max child nodes are 2



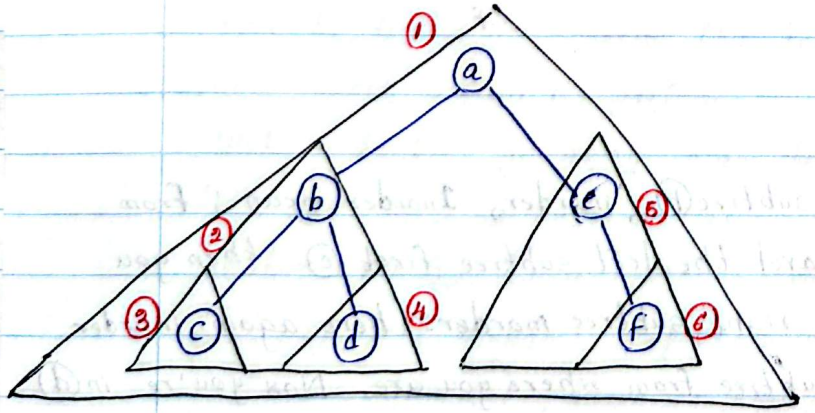
Binary Tree Traversal

* Without missing any node, visiting every node once & only once.

Preorder Traversal

mean: again
- Visit root in left subtree.
- Left subtree.
- Right subtree.

* 3 steps :-
  - Visit the root
  - Traversal the left subtree in preorder.
  - Traverse the right subtree



a
b
c
d
e
f

In step ③ (in c subtree), we visit left of c, since it is not there (no left subtree of ©, we go again to root of © which is ⓑ) Then we perform the traversal again.

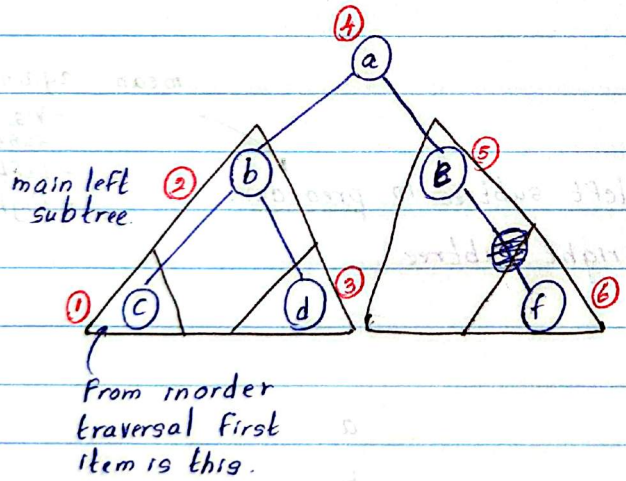
ⓑ, already visited; then left © : already visited. then right ⓓ write it down. Then visit the left of ⓓ, since no left go back to roots and perform this. untill its done.

InOrder Traversal

* Visiting happen in the middle.

3 orders : • ~~left sl~~ Left subtree traversal in inorder
• Visit the root.
• Traverse the right subtree in inorder.

mean; • First traverse the left subtree
• Visit root
• Right traversal.



c
b
d
a
e
f

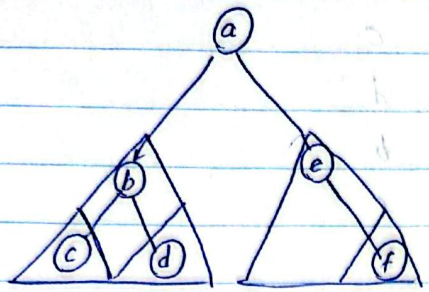
Here first traverse the left subtree ⓑ, inorder; Inorder mean; from where you are you should travel the left subtree first © then you visit the root ⓑ. Later the right subtree inorder.. here again inorder mean you traverse the left subtree from where you are. Now you're in ⓓ since there is no left subtree to ⓓ, you visit the root which is ⓓ then right it. Then traverse the right subtree. No right subtree.

So go like this.

* Here if ⓔ had another left subtree, we have write it down before ⓔ.
Why → Here the first step is the left traversal.

EDUCATION IS THE BEST INVESTMENT

CamScanner

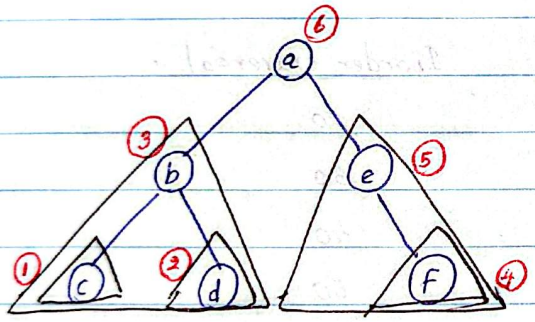


c
b
d
a
e
f

PostOrder Traversal

- Visiting happen last.

Algorithm (3 steps) :
- Left traversal in postorder. ← means! go again in these orders.
- Right traversal in postorder.
- Visit the root.



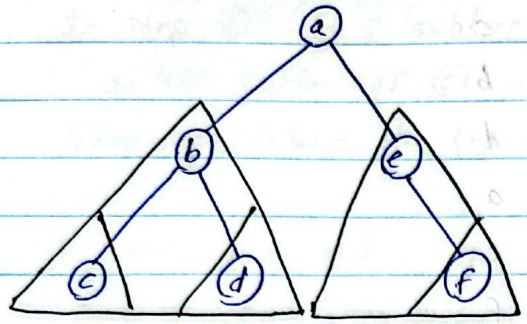
c
d
b
f
e
a

Here the first, we traverse the left subtree the main subtree. From there again we perform left traversal. The next left subtree with node ©. go again from left traversal. © node ; look for the left subtree ; no subtree. Then traverse right subtree. No right subtree for ©. Then visit the root. which is © . Write it.
Then © left subtree completed. Then next level. We start the postorder again. (b) , there we traversal the left subtree. – already done. Then traverse right subtree. Which is (d). Then again perform postorder. No left, no right. Then visit root which is (d) . Then that level complete go to next level (e) right subtree.

+ If (e) had a left subtree we right it before (f)

EDUCATION IS THE BEST INVESTMENT

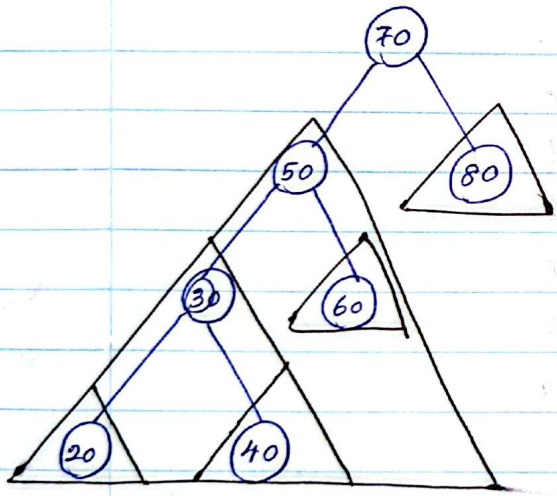
CamScanner



c
d
b
f
e
a

Binary search tree

- All left subtree nodes contain values less than the root.
- All right subtree nodes contain values equal or greater than root.
- All the subtree should follow the same roots as above (all subtree must binary search tree.)



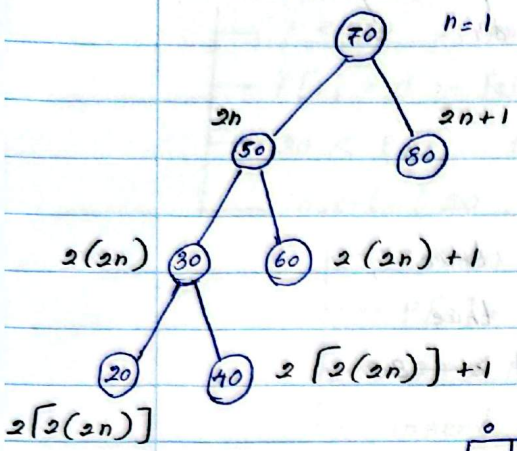
Inorder traversal

20
30
40
50
60
70
80

No.

Date. 18/09/2026

Array based tree



* Parent = n
* Left child = 2n
* Right child = 2n+1

* Like this we can put this into a 2D array.

0	1	2	3	4	5	6	7	8	9	10	11
	70	50	80	30	60			20	40		

* Some elements may remain empty unless we have a very balanced tree.

12 ← size

0	1	2	3	4	5	6	7	8	9	10	11
0	~~70~~	~~50~~	~~80~~	~~30~~	~~60~~	0	0	~~20~~	~~40~~	0	0

Initialize with 0

① 70 (data), 70 (d), 1 (n) — default value.

② 50 (data), 50 (data), 1 (n)

ex: 1) Insert (70, 1)

- n < size → true
- P[n] = 0 → true
- P[n] = 70
- return true.

Ex: 2) insert (50, 1)

- 1 < size - true
- P[1] = 0 - false ; P[n] = 70.
- 50 < P[1] - true.
- insert (50, 2)
  - 2 < 50 - true.
  - P[2] = 0 - true
  - P[2] ← 50
  - return true.

③ 50 (d), 2 (n)

After returned to the first call these parameters will be deleted.

EDUCATION IS THE BEST INVESTMENT

CamScanner

Ex 3;)

80 (data)

insert (80, 1) — 80 (d), 1 (n)

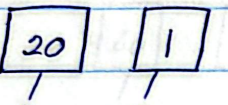
- 1 < 80 - true
- P[1] = 0 false.
- 80 < P[1] false.
- insert (80, 3)
  - 3 < 80 - true
  - P[3] = 0 - true
  - ~~80 <~~ P[3] ← 80
  - return true.

Ex: 4)

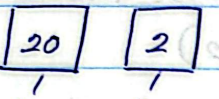
insert (30, 1) — 30 (d), 1 (n)

- 1 < 30 - true.
- P[1] = 0 - false.
- 30 < P[1] - true.
- insert (30, 2) — 30 (d), 2
  - 2 < 30 - true
  - P[2] = 0 - false
  - 30 < P[2] - true
  - insert (30, 4) — 30, 4.
    - 4 < 30 - true
    - P[4] = 0 - true
    - P[4] ← 30
    - return true.

5) insert (20, 1)



- 1 < 20 - true
- P[1] = 0 - false
- 20 < P[1] - true
- insert (20, 2)

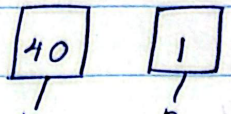


  - 2 < 20 - true
  - P[2] = 0 - False
  - 20 < P[2] - true
  - insert (20, 4)

    - 4 < 20 - true
    - P[4] = 0 - False
    - 20 < P[4] - true
    - insert (20, 8)

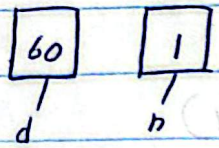
      - 8 < 20 - true
      - P[8] = 0 - true
      - P[8] ← 20
      - return true.

6) insert (40, 1)

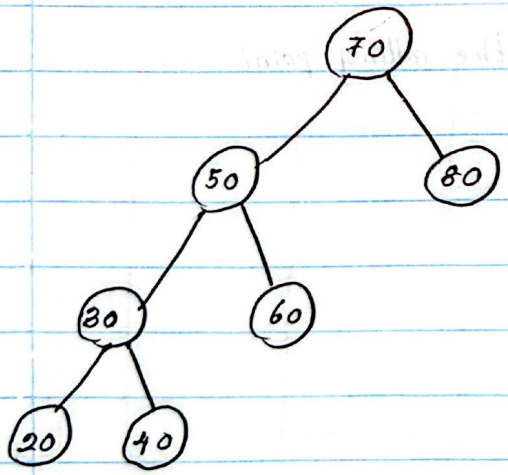


- 1 < 40 - true
- P[1] = 0 False
- 40 < P[1] ~~false~~ true.
- insert (40, 2)
  - 2 < 40 - true
  - P[2] ⇐ 0 - False
  - 40 < P[2] - true.
  - insert (40, 4)
    - 4 < 40 - true.
    - P[4] = 0 - false
    - 40 < P[4] - false.
    - insert (40, 9)
      - 9 < 40 - true
      - P[9] = 0
      - P[9] ← 40
      - return true.

7) insert (60, 1)



- 1 < 60 - true
- P[1] = 0 - false
- 60 < P[1] - true
- insert (60, 2)
  - 2 < 60 - true
  - P[2] = 0 - false
  - 60 < P[2] - false
  - insert (60, 5)
    - 5 < 60 - true
    - P[5] = 0 - true
    - P[5] ← 60
    - return true.



CamScanner

traverseInorder (1)
- 1 < size - true
- P[1] ≠ 0 - true
- traverseInorder (2)
  - 2 < size - true
  - P[2] ≠ 0 - true
  - traverseInorder (4)
    - 4 < size - true
    - P[4] ≠ 0 - true
    - traverseInorder (8)
      - 8 < size - true
      - P[8] ≠ 0 - true
      - traverseInorder (16)
        - 16 < size - false
        - return to the calling point
      - Print P[8] → 20
      - traverseInorder (17)
        - 17 < size - false
        - return
    - Print P[4] → 30
    - traverseInorder (9)
      - 9 < size - true
      - P[9] ≠ 0 - true
      - traverseInorder (18)
        - 18 < size - false
        - return
    - Print P[9] → 40
    - traverseInorder (19)
      - 19 < size - false
      - return
  - Print P[2] → 50
  - traverseInorder (5)

No. .....

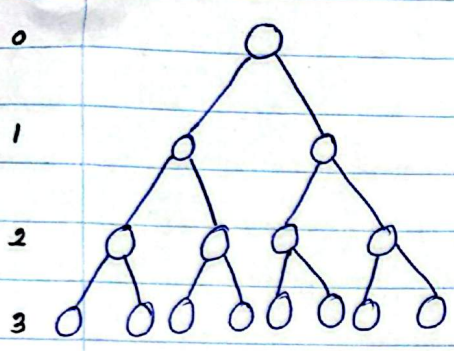
- 5 < size - true
- P[5] ≠ 0 - true
- traverseInorder (10)
  - 10 < size - true
  - P[10] ≠ 0 - false
  - return
- Print P[5] → 60
- traverseInorder (11)
  - 11 < size - true
  - P[11] ≠ 0 - false
  - return

- Print P[1] → 70
- traverseInorder (3)
  - 3 < size - true
  - P[3] ≠ 0 - true
  - traverseInorder (6)
    - 6 < size - true
    - P[6] ≠ 0 - ~~true~~ false.
    - ~~traverseInorder(11)~~ return.
      - ~~11 < size - true~~
      - ~~P[11] ≠ 0 - false~~
      - ~~return~~
- Print P[3] → 80
- traverseInorder (7)
  - 7 < size - true
  - P[7] ≠ 0 - false
  - return

No. Date.

Linked list based tree.

Flexible
More memory efficient.



hight (h)	nodes	total	
0	1	1	$2^{0+1}-1=1$
1	2	1+2=3	$2^{1+1}-1=3$
2	4	1+2+4=7	$2^{2+1}-1=7$
3	8	1+2+4+8=15	$2^{3+1}-1=15$
⋮	⋮	⋮	

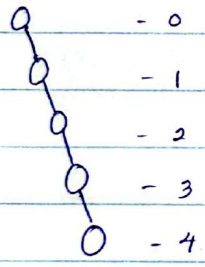
$$2^{h+1}-1$$

He need 2^{h+1} ~~nodes~~ elements to accomadate the tree.

nodes - $2^{h+1}-1$

Ex! For 4 levels He need 16 array elements & 15 nodes.

*. IF the tree is unbalanced & hight increase, the memory wastage increase dramatically.



Disadvantages of array based tree.

- Cannot grow.
- Cannot shrink
- Waste would dramatically

Solution ⟶ linked list based tree.

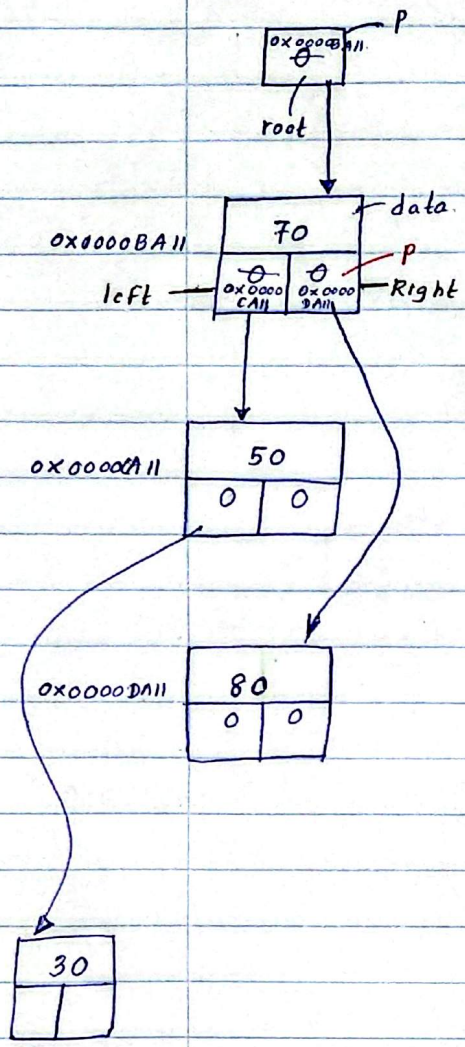
- Can grow
- Can shrink

If you don't know the hight, not balanced ⟶ go with linked list based

No.

Date. 25/09/2026

Linked list-based tree



① addNode (root, 70)

- P = 0 : true.
- P → data ← 70. create node & store 70.

70 (d) 70 (data)

② addNode (root, 50)

- P = 0 : false.
- 50 < 70 : true
- addNode (70.left, 50)
  - P = 0 : ~~false~~ true.
  - create node & store 50

50 (d) 50 (data.)

③ addNode (root, 80)

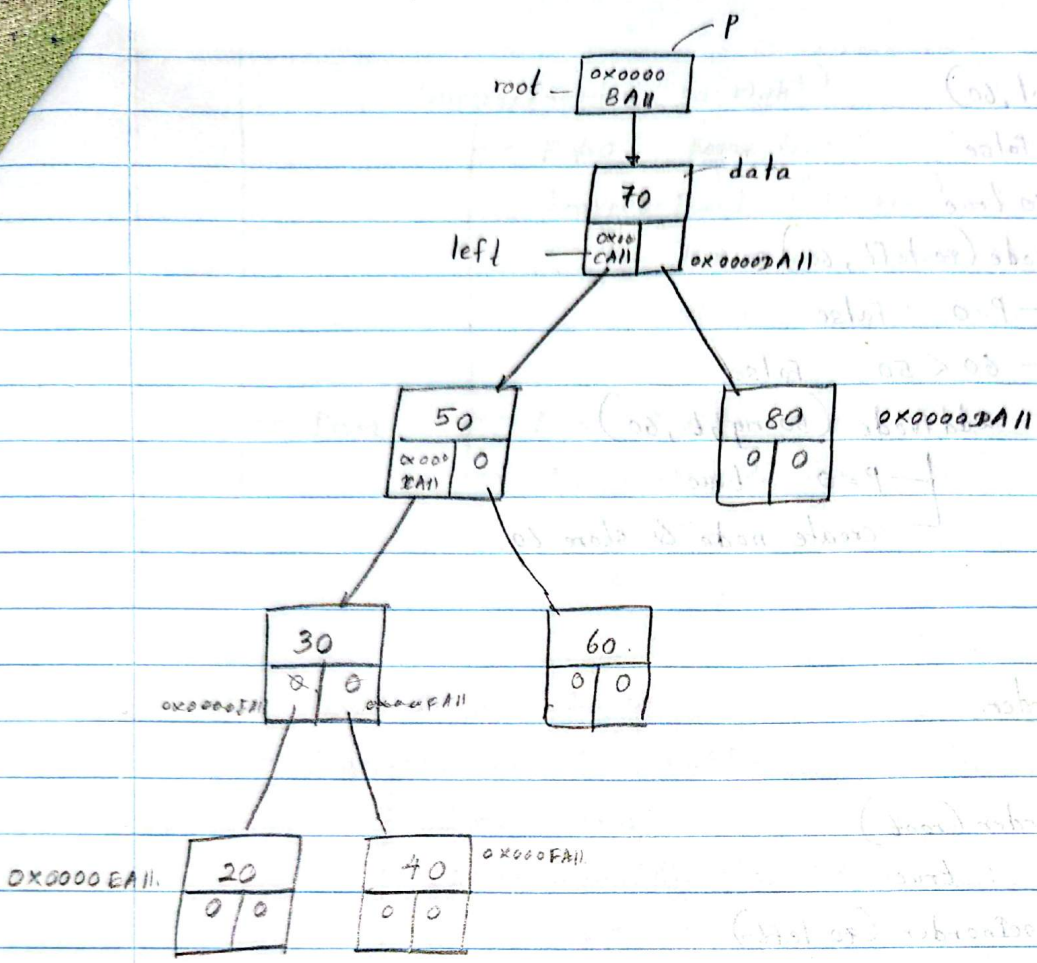
- P = 0 : false.
- 80 < 70 : false.
- addNode (P.right, 80)
  - P = 0 : true
  - create node & store 80.

80 (d) 50. (data)

④ addNode (root, 30)

- P = 0 : false
- 30 < 70 : true
- addNode (P.left, 30)
  - P = 0 : false
  - ~~Traverse~~ Intraa.
  - 30 < 50 : true.
  - addNode (50.left, 30)
    - P = 0 : true
    - create node & store 30.

30 (d) 30 (data)



⑤ addNode (root, 20)
- P=0 : false
- 20 < 70 : true
- addNode (P.left, 20)
  - P=0 : false
  - 20 < 30 : true
  - addNode (30left, 20)
    - P=0 true
    - create node & store. 20

⑥ addNode (root, 40)
- P=0 : false
- 40 < 70 : true
- addNode (P.left, 40)
  - P=0 : false
  - 40 < 50 : true.
  - addNode (P.left, 40)
    - P=0 : false
    - 40 < 30 : false
    - add Node (P.right, 40)
      - P=0 : true
      - create node & store 40.

EDUCATION IS THE BEST INVESTMENT

CamScanner

No. .....

⑦ addNode (root, 60)

- P=0 : false
- 60 < 70 true
- addNode (70.left, 60)
  - P=0 : false
  - 60 < 50 : False
  - addNode (50.right, 60)
    - P=0 : true
    - create node & store 60.

traverseInorder.

traverseInorder (root)

- P≠0 : true.
- traverseInorder (70.left)
  - P≠0 : true
  - traverseInorder (50.left)
    - P≠0 : true
    - traverseInorder (30.left)
      - P≠0 : true.
      - traverseInorder (20.left)
        - P≠0 : false.
        - return
      - Print P.data ⟶ 20
      - traverseInorder (20.right)
        - P≠0 : false
        - return
    - Print P.data ⟶ 30
    - traverseInorder (30.right)

EDUCATION IS THE BEST INVESTMENT

CamScanner

traverseInorder (30.right)
- P≠0 : true
- traverseInorder (40.left)
  - P≠0 : false
  - return
- Print P.data → 40
- traverseInorder (40.right)
  - P≠0 : false
  - return

Print P.data → 50
traverseInorder (50.right)
- P≠0 : true
- traverseInorder (60.left)
  - P≠0 : false
  - return
- Print P.data → 60
- traverseInorder (60.right)
  - P≠0 : false
  - return

Print P.data → 70
traverseInorder (70.right)
- P≠0 : true
- traverseInorder (80.left)
  - P≠0 : false
  - return
- Print P.data → 80
- traverseInorder (80.right)
  - P≠0 : false
  - return

EDUCATION IS THE BEST INVESTMENT

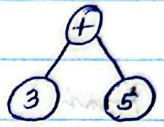
CamScanner

No. Date.

Tree Applications

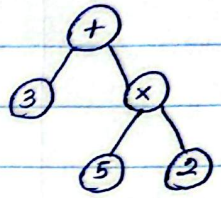
- Represent hierarchically related data.
  - Maintan folders / files

- Represent mathematical expressions.
  - We call them expression trees.
    - Parent node → Operator
    - Child nodes → Operands



3+5

3+5×2



- Data compression & encoding.

- Efficiently search dynamic sets of data.

  If we want to insert / remove data only thing have to do is change the address. Very efficient.
  - In arrays all elements will have to shift for very small changes.

Recomended References